Лабораторна робота №5
Передача даних між HTML-формою та PHP-сценарієм

Крайняк Михайло Юрійович, 451 група

1. Тема, мета і середовище

Мета: навчитися отримувати й обробляти дані HTML-форм засобами PHP, перевіряти введення та зрозуміло повідомляти результати; закріпити роботу з асоціативними масивами, підключенням файлів і метаданими завантажень.

PHP вебсервера	**8.2.12** (заголовок `X-Powered-By: PHP/8.2.12`, XAMPP). PHP у терміналі той самий. OSPanel і PHP 8.1 не встановлені. Код використовує лише можливості PHP 8.1 (`match`, `str_*`), тож на 8.1 працює так само.
Розширення	`mbstring` (для `mb_strlen()` і `mb_check_encoding()`), `filter` (`FILTER_VALIDATE_EMAIL`), вбудовані функції `password_*`. Усі доступні.
Ліміти завантаження	`file_uploads = 1`, `upload_max_filesize = 40M`, `post_max_size = 40M`, `max_file_uploads = 20` (типові для XAMPP). Ліміти дозволяють передати файл більший за 1 МіБ, тож межу 1 МіБ перевіряє код. Для сценаріїв перевищення лімітів запущено ще два екземпляри сервера з тими самими файлами: з `-d post_max_size=2M` (порт 8051) і з `-d upload_max_filesize=512K` (порт 8052). `php.ini` не змінювався.
Адреса запуску	`http://127.0.0.1:8050/lab-05/` — вбудований сервер PHP. Корінь сервера містить каталог `lab-05`, як `C:\xampp\htdocs\lab-05`. Сторінки відкривалися лише через HTTP, усі файли в UTF-8.
Інструкція запуску	Скопіювати вміст `src` у `htdocs/lab-05` і відкрити `http://localhost/lab-05/`. Інший варіант: із каталогу, де `lab-05` вказує на `src`, виконати `php -S 127.0.0.1:8050`. Перевірки — `bash tools/checks.sh`.

2. Створені файли

Файл	Призначення
`src/stage2/index.php`, `src/stage2/login.php`	Крок 2: форма з методички без змін і окремий обробник: метод, типи, непорожність, «Дані отримано для перевірки»
`src/stage2/query.php`	Тимчасова GET-форма з нейтральним полем `query` для порівняння з POST
`src/index.php`	Кроки 3–4: керування запитом входу. Перевірки, пошук користувача, `password_verify()`, підключення даних і шаблону
`src/login-form.php`	Шаблон форми входу: помилки, збережений логін, привітання
`src/users.php`	Масив трьох вигаданих користувачів (логін → ім’я, bcrypt-хеш); повертається через `return`
`src/feedback.php`, `src/feedback-form.php`	Самостійне завдання: обробка форми зворотного зв’язку з вкладенням і її шаблон

Файл	Призначення
`src/helpers.php`, `src/style.css`	Спільні функції (екранування з заготовки, перевірка втрати тіла POST, формат розміру, шапка) і стилі
`given/src/index001.php`	Початкова заготовка з методички, без змін
`tools/make_hash.php`	Одноразове створення bcrypt-хешу. Пароль читається зі стандартного введення, тож у коді й історії команд його немає
`tools/checks.sh` → `tools/checks-output.txt`, `tools/multipart_body.py`	Автоматичні перевірки (curl) і побудова сирих тіл multipart для сценаріїв, які `curl -F` не відтворює
`results/w3c-validation.txt`	Результат перевірки згенерованого HTML валідатором W3C

3. GET і POST, валідація, екранування, перевірка хешу

GET і POST. Атрибут `name` поля стає ключем у даних запиту, `id` лише зв'язує поле з `label`.

- **GET-форма** (`query.php`) кладе дані в рядок URL: `…/query.php?query=%D1%80%D0%BE…` (UTF-8, URL-кодування). Тіла запиту немає, PHP читає значення з `$_GET['query']`. Значення лишається в історії браузера й журналах сервера.
- **POST-форма** входу передає `login=…&password=…` у тілі (`Content-Type: application/x-www-form-urlencoded`), PHP читає їх з `$_POST`.
- **Форма з файлом** використовує `enctype="multipart/form-data"`: кожне поле стає окремою частиною тіла з `Content-Disposition: form-data; name="…"`, а файл — частиною з `filename` і `Content-Type`.

POST дані не шифрує: в тілі вони так само відкриті, захищає лише HTTPS (рис. 1).

POST /lab-05/stage2/login.php — дані в тілі запиту (Form Data)

```
POST /lab-05/stage2/login.php HTTP/1.1
Host: 127.0.0.1:8050
Content-Length: 38
Content-Type: application/x-www-form-urlencoded

Тіло: login=student_1&password=••••••••  (значення приховано у звіті)
```

Відповідь

```
HTTP/1.1 200 OK
Host: 127.0.0.1:8050
Date: Thu, 29 Oct 2026 07:22:34 GMT
```

GET /lab-05/stage2/query.php — дані в рядку URL (Query String)

```
GET /lab-05/stage2/query.php?
query=%D1%80%D0%BE%D0%B7%D0%BA%D0%BB%D0%B0%D0%B4+%D0%B0%D0%B2%D1%82%D0%BE%D0%B1%D1%83%D1%81%D1%96%D0%B2 HTTP/1.1
Host: 127.0.0.1:8050

Тіло запиту: немає
```

Відповідь

```
HTTP/1.1 200 OK
Host: 127.0.0.1:8050
Date: Thu, 29 Oct 2026 07:22:34 GMT
```

POST /lab-05/feedback.php — multipart/form-data: заголовки частин

```
Content-Type: multipart/form-data; boundary=------------------------yQUe84pgaN5xV02WqgSI5h
Content-Disposition: form-data; name="name"
Content-Disposition: form-data; name="email"
Content-Disposition: form-data; name="message"
Content-Disposition: form-data; name="attachment"; filename="rozklad.txt"
Content-Type: text/plain
```

Зібрано з обміну curl -v / --trace-ascii з тим самим сервером; назви полів login, password, query, name, email, message, attachment — значення атрибутів name. Пароль у звіт не потрапляє.

Рис. 1. Метод, адреса й назви полів у запитах: POST (дані в тілі), GET (дані в URL), multipart/form-data (частини з `name`, файл — з `filename` і `Content-Type`). Пароль у знімку приховано.

Серверна валідація виконується в такому порядку:

1. метод (`$_SERVER['REQUEST_METHOD']`);
2. наявність і тип поля (`?? null`, потім `is_string()`: `login[]=x` дає масив, на якому `trim()` чи `htmlspecialchars()` спричинили б TypeError);
3. нормалізація: `trim()` для логіна, імені, пошти й повідомлення, але не для пароля;
4. обов’язковість (явне `=== ''`, бо рядок «0» не порожній);
5. формат і межі.

HTML-атрибути `required` і `maxlength` лише допомагають користувачу, а всі перевірки повторено на сервері: curl їх обходить.

Екранування. Будь-яке значення від користувача виводиться через `escapeHtml()`, тобто `htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`. Це стосується логіна, імені, пошти, повідомлення й назви файла. Тому `<b>Перевірка</b>` показується як текст (рис. 7), а розмітка в назві файла не виконується.

Перевірка хешу. У `users.php` зберігаються лише bcrypt-хеші. Їх один раз створено командою `tools/make_hash.php` (`password_hash($password, PASSWORD_BCRYPT)`). Під час входу послідовно:

1. перевіряються логін (3–30 символів `[A-Za-z0-9_]`) і довжина пароля 8–72 *байти*: `strlen()`, бо bcrypt враховує не більше 72 байтів, а українська літера займає 2;
2. шукається запис `$users[$login]`;
3. `password_verify()` порівнює пароль із хешем; сіль і вартість хешу записані в самому рядку `$2y$10$…`.

Для невідомого логіна перевіряється фіктивний хеш, тож час відповіді не підказує, чи існує логін. Повідомлення однакове: «Неправильний логін або пароль».

```
<?php
declare(strict_types=1);
// Кроки 3–4: обробка входу в тому самому файлі. Тут лише керування запитом;
// дані — users.php, розмітка форми — login-form.php (фіксовані шляхи від __DIR__).
require __DIR__ . '/helpers.php';
$users = require __DIR__ . '/users.php';   // обов’язкові дані: без них сторінка не має сенсу → require

// Хеш для відсутнього логіна: password_verify() виконується завжди, тож час відповіді
// не підказує, чи існує логін. Хеш випадкового пароля, який ніде не збережено.
const DUMMY_HASH = '$2y$10$6jxAfbRsnbiU1wzxmGsYy.pZKNoY1eCEWEj/siZUJ6SZ/Khf689vS';
const LOGIN_PATTERN = '/^[A-Za-z0-9_]{3,30}$/D';

$errors = [];
$login = '';
$userName = null;

if (postBodyLost()) {
    $errors[] = 'Дані форми не отримано: запит, імовірно, перевищив post_max_size.';
} elseif (isPost()) {
    $rawLogin = $_POST['login'] ?? null;
    $password = $_POST['password'] ?? null;

    if (!is_string($rawLogin) || !is_string($password)) {
        $errors[] = 'Логін і пароль мають бути рядками.';
    } else {
        $login = trim($rawLogin);          // логін нормалізуємо, пароль — ні: пробіли можуть бути його
частиною
        if ($login === '' || $password === '') {
            $errors[] = 'Заповніть логін і пароль.';
        } else {
            // Правила формату — до пошуку користувача й перевірки хешу.
            if (!preg_match(LOGIN_PATTERN, $login)) {
                $errors[] = 'Логін: від 3 до 30 символів — латинські літери, цифри або «_».';
            }
            $bytes = strlen($password);    // bcrypt враховує не більше 72 байтів
            if ($bytes < 8 || $bytes > 72) {
                $errors[] = 'Пароль: від 8 до 72 байтів (українська літера займає 2 байти).';
            }
        }
        if ($errors === []) {
            $user = $users[$login] ?? null;
            $valid = password_verify($password, $user['password_hash'] ?? DUMMY_HASH);
            if ($user !== null && $valid) {
                $userName = $user['name'];
            } else {
                $errors[] = 'Неправильний логін або пароль.';   // однаково для невідомого логіна й хибного
пароля
            }
        }
    }
} elseif (($_SERVER['REQUEST_METHOD'] ?? 'GET') !== 'GET') {
    header('Allow: GET, POST');
    http_response_code(405);
    exit('Метод не підтримується.');
}
```

```
if ($errors !== [] && isPost()) {
    http_response_code(postBodyLost() ? 413 : 422);
}

require __DIR__ . '/login-form.php';   // шаблон: використовує $errors, $login, $userName
```

```
<?php
declare(strict_types=1);
// Вигадані навчальні користувачі: ключ — логін, запис — ім’я та bcrypt-хеш пароля.
// Хеші створено один раз командою php tools/make_hash.php; самі паролі тут не зберігаються.
return [
    'olena_k' => [
        'name' => 'Олена Коваленко',
        'password_hash' => '$2y$10$UYXiQuysPobe/HggObhBb.JQmtTHcx2RIRP4jibJ9AiM7Td4rKdaK',
    ],
    'taras_m' => [
        'name' => 'Тарас Мельник',
        'password_hash' => '$2y$10$gI5AaY2RzKuobOg4MshypuAcOS4jVB3DegWSGJfFYVnNG3LznE602',
    ],
    'iryna_99' => [
        'name' => 'Ірина Бондар',
        'password_hash' => '$2y$10$98SDXJMQrkk7fcCKCyxfZux/oI21cYipEZGyIbD2hGiziuCPYe0.K',
    ],
];
```

Чому `require`. `users.php`, `helpers.php` і шаблон обов’язкові: без них сторінка не має сенсу. `require` зупиняє виконання, якщо файла немає, а `include` лише видав би попередження, і код працював би з невизначеними даними. Шляхи фіксовані й будуються від `__DIR__`, тому не залежать від поточного каталогу й не містять даних користувача.

4. Хід роботи зі знімками

Крок 2. Форма й окремий обробник

```
if (($_SERVER['REQUEST_METHOD'] ?? '') === 'POST') {
    $rawLogin = $_POST['login'] ?? null;
    $password = $_POST['password'] ?? null;
    if (!is_string($rawLogin) || !is_string($password)) {
        $errors[] = 'Логін і пароль мають бути рядками.';
    } elseif (trim($rawLogin) === '' || $password === '') {
        $errors[] = 'Заповніть логін і пароль.';
    } else {
        $login = trim($rawLogin);
    }
    if ($errors !== []) {
        http_response_code(422);
    }
}
```

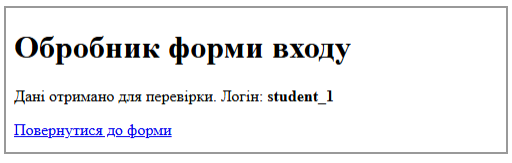


Рис. 3. Коректний POST: екранований логін і «Дані отримано для перевірки».

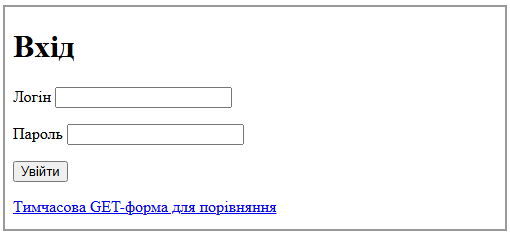


Рис. 2. Форма з методички (`action="login.php"`).

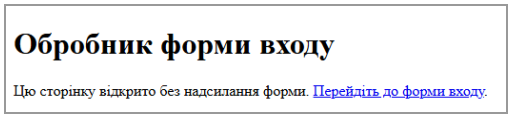


Рис. 4. GET до `login.php`: підказка перейти до форми, попереджень немає.



Рис. 5. Тимчасова GET-форма: значення `query` прийшло з URL.

Кроки 3–4. Обробка в тому самому файлі, масив користувачів, поділ на файли

Обробка перенесена в `index.php` (`action="index.php"`). Перше відкриття (GET) відокремлено від POST. Повідомлення збираються в `$errors` і показуються над формою. Логін після помилки повертається в поле, пароль — ні. Масив користувачів винесено в `users.php`, розмітку — в `login-form.php`. Після поділу всі перевірки повторено, і результати не змінилися (розділ 6).

```
<?php
// Шаблон форми входу. Очікує $errors (масив рядків), $login (рядок), $userName (рядок або null).
// Пароль у поле не повертається; усі значення екрануються.
pageHeader('Вхід', 'index.php');
?>
<p class="note">Навчальна перевірка логіна й пароля в межах одного запиту: сеанс входу не створюється.</p>
<?php if ($userName !== null): ?>
<p class="success" role="status">Вхід перевірено. Вітаємо, <?= escapeHtml($userName) ?>!</p>
<?php endif; ?>
<?php if ($errors !== []): ?>
<div class="errors" role="alert">
    <ul>
<?php foreach ($errors as $error): ?>
        <li><?= escapeHtml($error) ?></li>
<?php endforeach; ?>
    </ul>
</div>
<?php endif; ?>
<form action="index.php" method="post">
    <p>
        <label for="login">Логін</label>
        <input id="login" name="login" type="text" required maxlength="30" autocomplete="username"
               value="<?= escapeHtml($login) ?>" aria-describedby="login-hint">
        <span id="login-hint" class="hint">3–30 символів: латинські літери, цифри, «_»</span>
    </p>
    <p>
        <label for="password">Пароль</label>
        <input id="password" name="password" type="password" required autocomplete="current-password"
```

```
                aria-describedby="password-hint">
        <span id="password-hint" class="hint">8–72 байти</span>
    </p>
    <button type="submit">Увійти</button>
</form>
<?php
pageFooter();
```

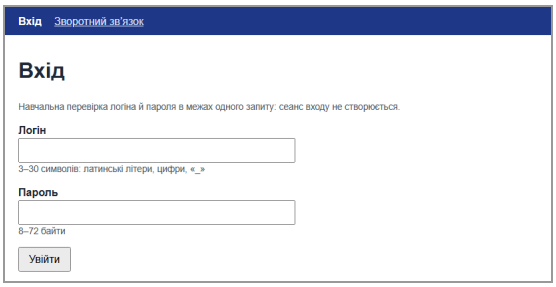


Рис. 6. Перше відкриття `index.php`: порожня форма без помилок і повідомлення про успіх.

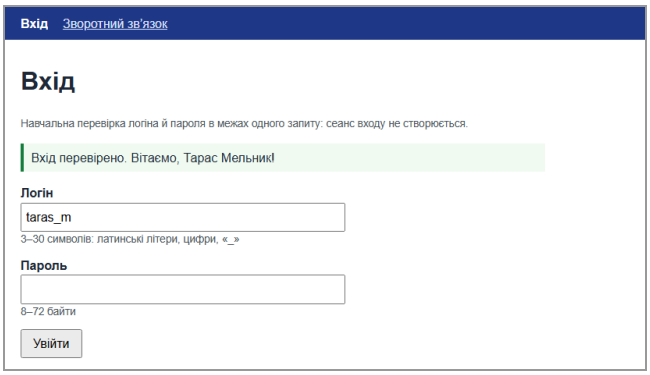


Рис. 7. Успішна перевірка для `taras_m`: екрановане ім'я «Тарас Мельник». Пароль цього користувача містить пробіл і не обрізається.

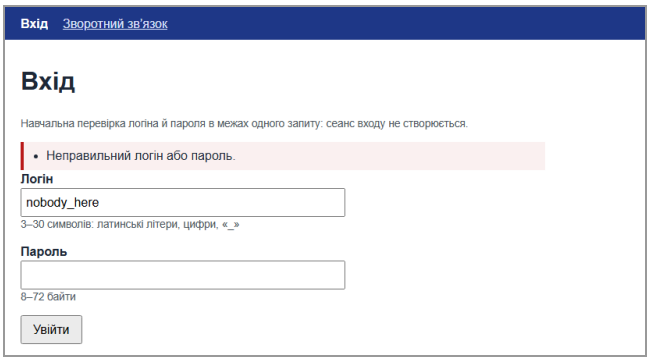


Рис. 8. Невідомий логін: «Неправильний логін або пароль» — те саме, що й для хибного пароля. Логін збережено, поле пароля порожнє.

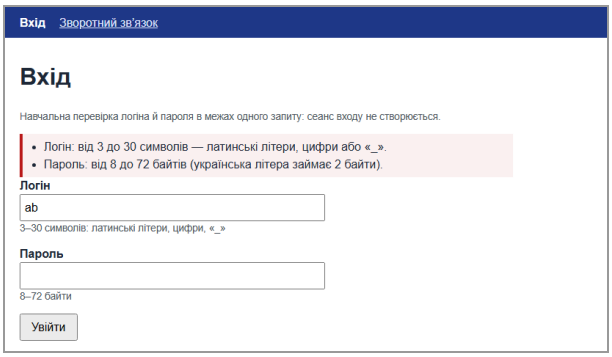


Рис. 9. Логін на 2 символи й пароль на 5 байтів відхилено до пошуку користувача: два зрозумілі повідомлення.

Крок 5. Форма зворотного зв'язку (самостійне завдання)

Обробка текстових полів:

- кожне поле проходить `textField()`: відсутнє поле, масив чи некоректний UTF-8 дають окреме повідомлення без попереджень, інакше значення обрізається `trim()`;
- довжини перевіряються `mb_strlen(…, 'UTF-8')` (ім'я до 100 символів, повідомлення 10–1000), пошта — `strlen()` до 254 байтів і `FILTER_VALIDATE_EMAIL`.

Вкладення перевіряє `checkAttachment()`:

1. структура `$_FILES['attachment']` (масив із цілим `error`; `attachment[]` відхиляється);

2. код `error`, зокрема `UPLOAD_ERR_NO_FILE`, коли вкладення немає — це не помилка;
3. типи `name`, `type`, `size`, `tmp_name` і `is_uploaded_file()`;
4. розмір від 1 байта до 1 МіБ.

Тимчасовий файл не переміщується й не читається. Показуються лише назва, розмір і MIME-тип із позначкою «заявлений клієнтом».

Помилки:

- помилка будь-якого поля, зокрема вкладення, блокує зведення (HTTP 422), а несекретні поля повертаються у форму;
- якщо POST має ненульову довжину, але `$_POST` і `$_FILES` порожні, PHP відкинув тіло через `post_max_size`. Тоді сторінка відповідає 413 із поясненням, а не порожньою формою чи «успіхом».

```
/** Текстове поле POST: [нормалізоване значення, помилка або null]. */
function textField(string $key): array
{
    $raw = $_POST[$key] ?? null;
    if ($raw === null) {
        return ['', 'Поле «' . FIELD_LABELS[$key] . '» не надіслано.'];
    }
    if (!is_string($raw)) {
        return ['', 'Поле «' . FIELD_LABELS[$key] . '» має бути рядком.'];
    }
    if (!mb_check_encoding($raw, 'UTF-8')) {
        return ['', 'Поле «' . FIELD_LABELS[$key] . '» містить некоректні символи (очікується UTF-8).'];
    }
    return [trim($raw), null];
}

/**
 * Перевірка вкладення: спершу структура й код error, далі типи ключів і розмір.
 * Повертає ['file' => null|['name', 'size', 'type'], 'error' => null|string]; file = null без помилки —
вкладення немає.
 */
function checkAttachment(): array
{
    if (!array_key_exists('attachment', $_FILES)) {
        return ['file' => null, 'error' => null];
    }
    $file = $_FILES['attachment'];
    if (!is_array($file) || !array_key_exists('error', $file) || !is_int($file['error'])) {
        return ['file' => null, 'error' => 'Некоректні дані вкладення: очікується один файл.'];
    }
    $uploadError = match ($file['error']) {
        UPLOAD_ERR_OK, UPLOAD_ERR_NO_FILE => null,
        UPLOAD_ERR_INI_SIZE => 'Файл перевищує серверну межу upload_max_filesize (' .
ini_get('upload_max_filesize') . ').',
        UPLOAD_ERR_FORM_SIZE => 'Файл перевищує межу, задану у формі.',
        UPLOAD_ERR_PARTIAL => 'Файл передано не повністю. Спробуйте надіслати ще раз.',
        UPLOAD_ERR_NO_TMP_DIR, UPLOAD_ERR_CANT_WRITE => 'Сервер не зміг тимчасово зберегти файл. Зверніться
до адміністратора.',
        UPLOAD_ERR_EXTENSION => 'Завантаження зупинено розширенням PHP на сервері.',
        default => 'Невідомий код помилки завантаження: ' . $file['error'] . '.',
    };
    if ($uploadError !== null) {
        return ['file' => null, 'error' => $uploadError];
    }
    if ($file['error'] === UPLOAD_ERR_NO_FILE) {
```

```
        return ['file' => null, 'error' => null];   // необов’язкове вкладення не вибрано — це не помилка
    }
    if (!is_string($file['name'] ?? null) || !is_string($file['type'] ?? null) || !is_int($file['size'] ??
null)
        || !is_string($file['tmp_name'] ?? null) || !is_uploaded_file($file['tmp_name'])) {
        return ['file' => null, 'error' => 'Некоректні дані вкладення.'];
    }
    if ($file['size'] < 1) {
        return ['file' => null, 'error' => 'Файл порожній (0 байт).'];
    }
    if ($file['size'] > ATTACHMENT_MAX_BYTES) {
        return ['file' => null, 'error' => 'Файл більший за 1 МіБ: ' . formatBytes($file['size']) . '.'];
    }
    // Тимчасовий файл не переміщуємо й не читаємо: PHP видалить його після запиту.
    return ['file' => ['name' => $file['name'], 'size' => $file['size'], 'type' => $file['type']], 'error'
=> null];
}
```

```
if (postBodyLost()) {
    $formError = 'Дані форми не отримано: розмір запиту ' . formatBytes((int) $_SERVER['CONTENT_LENGTH'])
        . ' перевищує post_max_size (' . ini_get('post_max_size') . '). Зменште вкладення й надішліть форму
ще раз.';
    http_response_code(413);
} elseif (isPost()) {
    foreach (array_keys($values) as $key) {
        [$values[$key], $error] = textField($key);
        if ($error !== null) {
            $fieldErrors[$key] = $error;
        }
    }
    if (!isset($fieldErrors['name'])) {
        $length = mb_strlen($values['name'], 'UTF-8');
        if ($length === 0) {
            $fieldErrors['name'] = 'Вкажіть ім’я.';
        } elseif ($length > NAME_MAX_CHARS) {
            $fieldErrors['name'] = 'Ім’я — до ' . NAME_MAX_CHARS . " символів (зараз $length).";
        }
    }
    if (!isset($fieldErrors['email'])) {
        if ($values['email'] === '') {
            $fieldErrors['email'] = 'Вкажіть електронну пошту.';
        } elseif (strlen($values['email']) > EMAIL_MAX_BYTES) {
            $fieldErrors['email'] = 'Адреса — до ' . EMAIL_MAX_BYTES . ' байтів.';
        } elseif (filter_var($values['email'], FILTER_VALIDATE_EMAIL) === false) {
            $fieldErrors['email'] = 'Некоректна адреса електронної пошти (приклад: name@example.com).';
        }
    }
    if (!isset($fieldErrors['message'])) {
        $length = mb_strlen($values['message'], 'UTF-8');
        if ($length < MESSAGE_MIN_CHARS || $length > MESSAGE_MAX_CHARS) {
            $fieldErrors['message'] = 'Повідомлення — від ' . MESSAGE_MIN_CHARS . ' до ' .
MESSAGE_MAX_CHARS
                . " символів без крайніх пробілів (зараз $length).";
        }
    }
    $attachment = checkAttachment();
    if ($attachment['error'] !== null) {
        $fieldErrors['attachment'] = $attachment['error'];
    }
    if ($fieldErrors === []) {
        $summary = $values + ['attachment' => $attachment['file']];   // успіх — лише після всіх перевірок
    } else {
        http_response_code(422);
    }
} elseif (($_SERVER['REQUEST_METHOD'] ?? 'GET') !== 'GET') {
    header('Allow: GET, POST');
    http_response_code(405);
    exit('Метод не підтримується.');
}
```

```
require __DIR__ . '/feedback-form.php';
```

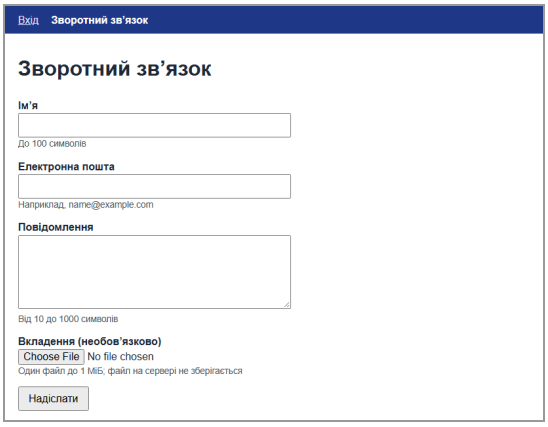


Рис. 10. `feedback.php` (GET): поля з підписами й підказками, `enctype="multipart/form-data"`.

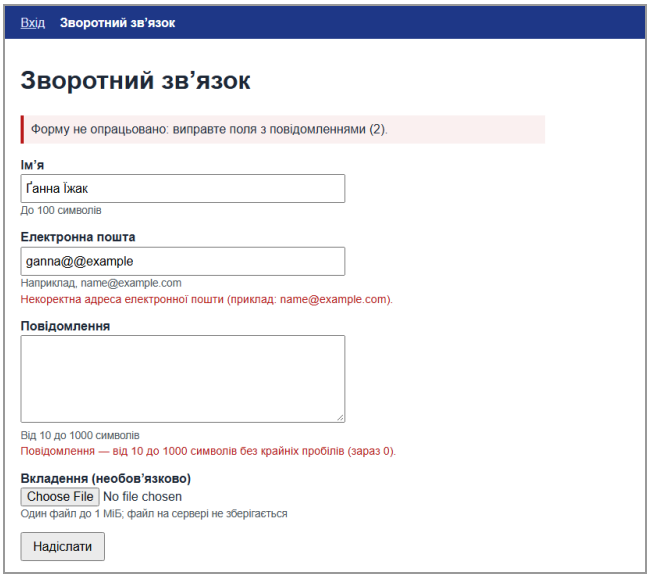


Рис. 11. Неправильна пошта й повідомлення з самих пробілів: помилки біля полів (зв'язані через `aria-describedby`), ім'я й пошту збережено.

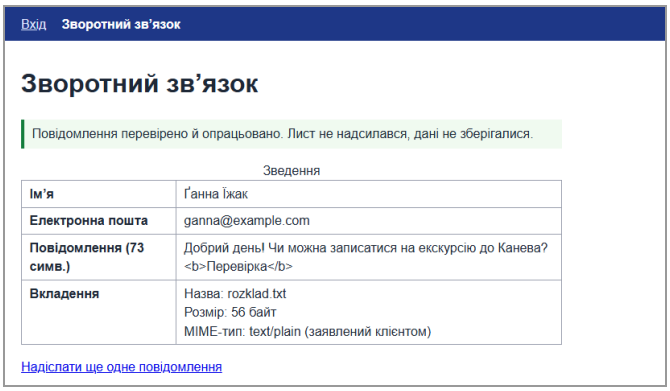


Рис. 12. Зведення: українське ім'я, підрахунок довжини, `<b>Перевірка</b>` як текст, метадані вкладення.

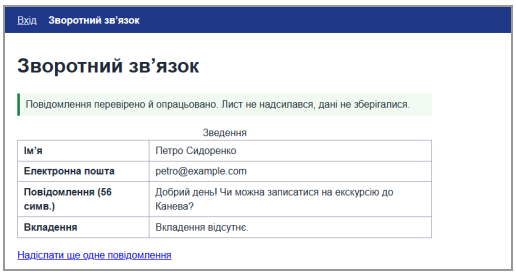


Рис. 13. Коректна форма без вкладення: «Вкладення відсутнє».

W3C Nu HTML Checker — результати для згенерованого HTML (29.10.2026)

Стан сторінки	Файл відповіді	Результат
Вхід — початкова сторінка (GET)	01_login_get.html	The document validates · 0 повідомлень
Вхід — невдалий вхід	02_login_fail.html	The document validates · 0 повідомлень
Вхід — помилки формату	03_login_format.html	The document validates · 0 повідомлень
Вхід — успішна перевірка	04_login_ok.html	The document validates · 0 повідомлень
Зворотний зв'язок — GET	05_feedback_get.html	The document validates · 0 повідомлень
Зворотний зв'язок — помилки полів	06_feedback_errors.html	The document validates · 0 повідомлень
Зворотний зв'язок — зведення з вкладенням	07_feedback_ok_file.html	The document validates · 0 повідомлень
Зворотний зв'язок — зведення без вкладення	08_feedback_ok_nofile.html	The document validates · 0 повідомлень
Крок 2 — форма	09_stage2_form.html	The document validates · 0 повідомлень
Крок 2 — GET login.php	10_stage2_get.html	The document validates · 0 повідомлень
Крок 2 — дані отримано	11_stage2_ok.html	The document validates · 0 повідомлень
Крок 2 — GET-форма query	12_query.html	The document validates · 0 повідомлень
Крок 2 — помилка типу	13_stage2_error.html	The document validates · 0 повідомлень

Перша перевірка 02/03 до виправлення: 1 error — Bad value “alert” for attribute “role” on element “ul” (розмітка з заготовки index001.php); виправлено обгорткою div role=alert.

Рис. 14. Перевірка згенерованого HTML валідатором W3C: 13 станів сторінок, 0 повідомлень. Перша перевірка знайшла помилку, яку виправлено (розділ 8).

5. Сценарії перевірки

№	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат	Висновок
1	Перше відкриття через GET	`index.php`, `feedback.php`, `stage2/login.php`	Порожні форми без помилок і успіху	200. Помилок 0, повідомлень про успіх 0; `login.php` — підказка «Перейдіть до форми входу»	**Виконано**
2	Правильні дані трьох користувачів	`olena_k`, `taras_m` (пароль із пробілом), `iryna_99` (пароль з українськими літерами)	Успіх із відповідним ім’ям	«Вітаємо, Олена Коваленко / Тарас Мельник / Ірина Бондар!»; пароля й хешу у відповіді немає	**Виконано**
3	Невідомий логін; неправильний пароль	`nobody_here`; `olena_k` + хибний пароль (також пароль із пробілом наприкінці, `OLENA_K`)	Однакове повідомлення	Скрізь 422 «Неправильний логін або пароль.»; логін збережено, поле пароля порожнє	**Виконано**
4	Відсутнє поле; порожнє значення; масив	Без `login` / `password`; порожні й з пробілів; `login[]`, `password[]`, `login[a][b]`; те саме для `name`, `email`, `message`	Зрозуміла відмова без warning, notice, TypeError	422: «Логін і пароль мають бути рядками.», «Заповніть логін і пароль.», «Поле «Ім’я» не надіслано / має бути рядком.»; службових повідомлень PHP немає (`display_errors=1`)	**Виконано**
5	Межі логіна й пароля	Логін 2, 3, 30, 31 символ; пароль 7, 8, 72, 73 байти; 36 і 37 українських літер	Крайні допустимі проходять валідацію, решта відхиляється	3, 30 символів і 8, 72 байти проходять валідацію (далі «Неправильний логін або пароль»). 2, 31 символ і 7, 73 байти відхилено повідомленням про формат. 36 літер = 72 байти — проходить, 37 = 74 — ні	**Виконано**

№	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат	Висновок
6	Українські літери в імені й повідомленні	«Ґанна Їжак», повідомлення 56 символів (102 байти)	Правильне відображення й підрахунок	Зведення показує ім’я й «Повідомлення (56 симв.)»	**Виконано**
7	Неправильна пошта; повідомлення з пробілів	`ganna@@example`; 14 пробілів	Помилка поля; несекретне введення збережено	«Некоректна адреса…», «… (зараз 0)»; ім’я й пошта повернулися в поля (рис. 11)	**Виконано**
8	Повідомлення 9, 10, 1000, 1001 символ	Літери «ї» (2 байти кожна)	Відхилення, прийняття, прийняття, відхилення	422 (зараз 9), 200, 200, 422 (зараз 1001). 10 символів з пробілами по краях — 200	**Виконано**
9	Розмітка в повідомленні	`<b>Перевірка</b> & <script>…`, ім’я `<i>Ганна</i>`	Звичайний текст	У відповіді `<b>Перевірка</b>`; тегів `<b>` і `<script>` немає (рис. 12)	**Виконано**
10	Коректна форма без вкладення	Тіло з `filename=""`, як у браузера; також без поля `attachment`	Успіх, «вкладення відсутнє»	200, «Вкладення відсутнє.» (рис. 13)	**Виконано**
11	Файл 1 байт, 1 МіБ, 1 МіБ + 1	1, 1 048 576, 1 048 577 байт	Перші два допустимі, третій ні	200 «1 байт», 200 «1,00 МіБ (1 048 576 байт)», 422 «Файл більший за 1 МіБ: … 1 048 577 байт»	**Виконано**
12	Порожній файл, коди помилок	0 байт; два файли `attachment[]`; 600 КіБ при `upload_max_filesize=512K`; обірване тіло multipart	Помилка без успішного підсумку	422: «Файл порожній (0 байт)», «очікується один файл», `UPLOAD_ERR_INI_SIZE` → «перевищує upload_max_filesize (512K)», `UPLOAD_ERR_PARTIAL` → «передано не повністю»; зведення немає	**Виконано**
13	POST понад `post_max_size`	Файл 3 МіБ при `post_max_size=2M`	Відмова з поясненням	413 «Дані форми не отримано: розмір запиту 3,00 МіБ … перевищує post_max_size (2M)…»; на `index.php` — аналогічне повідомлення. Перед HTML PHP вивів власне попередження «POST Content-Length … exceeds the limit», бо тестовий сервер працював із `display_errors=1` (див. нижче)	**Виконано**

№	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат	Висновок
14	Довжини імені й пошти, кодування	Ім’я 100/101 символ; пошта 254/255 байтів; `%C3%28` (некоректний UTF-8)	Межі як у таблиці завдання	100 і 254 — 200; 101 і 255 — 422 з повідомленням; некоректний UTF-8 — «містить некоректні символи»	**Виконано**
15	Валідація HTML	13 станів (форми, помилки, успіх, зведення)	Без помилок	0 повідомлень (рис. 14)	**Виконано**

Невиконаних сценаріїв немає. Що варто знати про окремі сценарії:

- **Як відтворено помилки завантаження.** `UPLOAD_ERR_INI_SIZE` і перевищення `post_max_size` отримано на окремих екземплярах сервера з меншими лімітами. `UPLOAD_ERR_PARTIAL` — сирим тілом multipart без завершальної межі.
- **Не відтворено.** Коди `UPLOAD_ERR_NO_TMP_DIR`, `CANT_WRITE`, `EXTENSION` залежать від стану сервера. Вони мають окремі повідомлення в `checkAttachment()`, але штучно не викликалися.
- **Попередження при `post_max_size`.** Його генерує ядро PHP до запуску сценарію, тому код не може його прибрати (оператор `@` тут не діє, та й не використовується). У робочому середовищі з `display_errors=0` воно потрапляє лише в журнал.

6. Вивід автоматичних перевірок

```
== 1. Середовище
PHP вебсервера: PHP/8.2.12
PHP CLI: 8.2.12; mbstring: так; file_uploads=1; upload_max_filesize=40M; post_max_size=40M;
max_file_uploads=20

== 2. Крок 2: форма й окремий обробник login.php
GET stage2/index.php: 200 / форма action=action="login.php" method="post"
GET stage2/login.php: 200 / Цю сторінку відкрито без надсилання форми.
POST коректні дані: 200 Дані отримано для перевірки. Логін:  student_1; пароль/хеш у відповіді: ні
POST login[]=x: 422 Логін і пароль мають бути рядками.
POST без password: 422 Логін і пароль мають бути рядками.
POST login з пробілів: 422 Заповніть логін і пароль.
POST login=<b>x</b>: 200 / у відповіді: &lt;b&gt;x&lt;/b&gt;
GET-форма query: адреса запиту http://127.0.0.1:8050/lab-05/stage2/query.php?
query=%D1%80%D0%BE%D0%B7%D0%BA%D0%BB%D0%B0%D0%B4+%D0%B0%D0%B2%D1%82%D0%BE%D0%B1%D1%83%D1%81%D1%96%D0%B2
Отримано з $_GET['query']: розклад автобусів

== 3. Вхід (index.php): перше відкриття і три користувачі
GET index.php: 200 / помилок: 0, успіху: 0, значення login: «»
olena_k: 200 Вхід перевірено. Вітаємо, Олена Коваленко!; пароль/хеш у відповіді: ні
taras_m: 200 Вхід перевірено. Вітаємо, Тарас Мельник!; пароль/хеш у відповіді: ні
iryna_99: 200 Вхід перевірено. Вітаємо, Ірина Бондар!; пароль/хеш у відповіді: ні
olena_k з пробілом наприкінці пароля: 422 Неправильний логін або пароль. — пароль не обрізається
" olena_k " (логін з пробілами): 200 Вхід перевірено. Вітаємо, Олена Коваленко! — логін нормалізовано
trim()

== 4. Невдалий вхід: однакове повідомлення
невідомий логін: 422 Неправильний логін або пароль.; login збережено: «nobody_here»; поле password порожнє:
так
неправильний пароль: 422 Неправильний логін або пароль.; пароль/хеш у відповіді: ні
ім’я в іншому регістрі (OLENA_K): 422 Неправильний логін або пароль.
```

```
== 5. Відсутнє поле, порожнє значення, масив
без login: 422 Логін і пароль мають бути рядками.
без password: 422 Логін і пароль мають бути рядками.
без полів (тіло x=1): 422 Логін і пароль мають бути рядками.
порожній login: 422 Заповніть логін і пароль.
порожній password: 422 Заповніть логін і пароль.
login з пробілів: 422 Заповніть логін і пароль.
login[]=olena_k: 422 Логін і пароль мають бути рядками.
password[]=x: 422 Логін і пароль мають бути рядками.
login[a][b]=1: 422 Логін і пароль мають бути рядками.
login=0 (рядок «0» не порожній): 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».
PUT index.php: 405 Метод не підтримується.

== 6. Межі довжини
логін 2 симв.: 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».
логін 3 симв.: 422 Неправильний логін або пароль.
логін 30 симв.: 422 Неправильний логін або пароль.
логін 31 симв.: 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».
пароль 7 байт: 422 Пароль: від 8 до 72 байтів (українська літера займає 2 байти).
пароль 8 байт: 422 Неправильний логін або пароль.
пароль 72 байт: 422 Неправильний логін або пароль.
пароль 73 байт: 422 Пароль: від 8 до 72 байтів (українська літера займає 2 байти).
пароль «Кіт12» (5 симв., 8 байт): 422 Неправильний логін або пароль.
пароль «Кіт1» (4 симв., 7 байт): 422 Пароль: від 8 до 72 байтів (українська літера займає 2 байти).
пароль 36 укр. літер (72 байти): 422 Неправильний логін або пароль.
пароль 37 укр. літер (74 байти, 37 симв.): 422 Пароль: від 8 до 72 байтів (українська літера займає 2
байти).
логін кирилицею «олена»: 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».
логін «olena-k»: 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».
логін з переносом рядка «olena_k\n»: 200 Вхід перевірено. Вітаємо, Олена Коваленко!  (trim прибирає \n)
логін «ol\nena»: 422 Логін: від 3 до 30 символів — латинські літери, цифри або «_».

== 7. Зворотний зв’язок: GET і коректні дані
GET feedback.php: 200 / помилок: 0, успіху: 0, enctype: enctype="multipart/form-data"
коректна форма без вкладення (файл не вибрано): 200 Повідомлення перевірено й опрацьовано. Лист не
надсилався, дані не зберігалися. |  Вкладення відсутнє.
   ім’я у зведенні: Ґанна Їжак; довжина повідомлення: Повідомлення (56 симв.) (mb_strlen = 56, strlen = 102
байт)
коректна форма без поля attachment (urlencoded): 200 Повідомлення перевірено й опрацьовано. Лист не
надсилався, дані не зберігалися.

== 8. Помилки полів і збереження введення
неправильна пошта: 422 email: Некоректна адреса електронної пошти (приклад: name@example.com).; ім’я
збережено: «Ґанна Їжак», пошта збережена: «ganna@@example»
повідомлення з пробілів: 422 message: Повідомлення — від 10 до 1000 символів без крайніх пробілів (зараз
0).
порожнє ім’я (пробіли): 422 name: Вкажіть ім’я.
ім’я 100 симв.: 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.
ім’я 101 симв.: 422 name: Ім’я — до 100 символів (зараз 101).
пошта 254 байт: 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.
пошта 255 байт: 422 email: Адреса — до 254 байтів.
усі поля порожні: 422 name: Вкажіть ім’я.|email: Вкажіть електронну пошту.|message: Повідомлення — від 10
до 1000 символів без крайніх пробілів (зараз 0).
без полів (лише x=1): 422 name: Поле «Ім’я» не надіслано.|email: Поле «Електронна пошта» не
надіслано.|message: Поле «Повідомлення» не надіслано.
name[]=x: 422 name: Поле «Ім’я» має бути рядком.
message[a]=x, email[]=y: 422 email: Поле «Електронна пошта» має бути рядком.|message: Поле «Повідомлення»
має бути рядком.
некоректний UTF-8 у name: 422 name: Поле «Ім’я» містить некоректні символи (очікується UTF-8).

== 9. Довжина повідомлення (українські літери «ї»)
9 симв. (18 байт): 422 message: Повідомлення — від 10 до 1000 символів без крайніх пробілів (зараз 9).
10 симв. (20 байт): 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.
1000 симв. (2000 байт): 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.
1001 симв. (2002 байт): 422 message: Повідомлення — від 10 до 1000 символів без крайніх пробілів (зараз
1001).
10 симв. з пробілами по краях (14 до trim): 200 Повідомлення перевірено й опрацьовано. Лист не надсилався,
дані не зберігалися.

== 10. Розмітка в повідомленні
```

```
<b>Перевірка</b>: тег <b> у відповіді: 0, екрановано: &lt;b&gt;Перевірка&lt;/b&gt; &amp;
&lt;script&gt;alert(1)&lt;/script&gt;; <script> у відповіді: 0; ім’я: &lt;i&gt;Ганна&lt;/i&gt;

== 11. Вкладення: 1 байт, 1 МіВ, 1 МіВ + 1
1 байт: 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.  | Назва: file-
1.txt; Розмір: 1 байт; MIME-тип: text/plain (заявлений клієнтом)
1048576 байт: 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не зберігалися.  | Назва:
file-1048576.txt; Розмір: 1,00 МіБ (1 048 576 байт); MIME-тип: text/plain (заявлений клієнтом)
1048577 байт: 422 attachment: Файл більший за 1 МіБ: 1,00 МіБ (1 048 577 байт). |
MIME-тип від клієнта (text .jpg як image/jpeg): 200 Повідомлення перевірено й опрацьовано. Лист не
надсилався, дані не зберігалися. | MIME-тип: image/jpeg (заявлений клієнтом)
назва файла з розміткою: 200 Повідомлення перевірено й опрацьовано. Лист не надсилався, дані не
зберігалися. | Назва: &lt;img src=x onerror=alert(1)&gt;.txt; tmp_name у відповіді: 0

== 12. Порожній файл і коди помилок завантаження
файл 0 байт: 422 attachment: Файл порожній (0 байт).; успіху: 0
attachment[] (два файли): 422 attachment: Некоректні дані вкладення: очікується один файл.
600 КіБ при upload_max_filesize=512K (8052): 422 attachment: Файл перевищує серверну межу
upload_max_filesize (512K).; успіху: 0
обірване тіло multipart: 422 attachment: Файл передано не повністю. Спробуйте надіслати ще раз.

== 13. POST понад post_max_size
3 МіБ при post_max_size=2M (8051), feedback.php: 413 Дані форми не отримано: розмір запиту 3,00 МіБ (3 146
373 байт) перевищує post_max_size (2M). Зменште вкладення й надішліть форму ще раз.
   попередження ядра PHP до запуску сценарію (видно лише через display_errors=1): POST Content-Length of
3146373 bytes exceeds the limit of 2097152 bytes in
tools/checks.sh: line 138: unexpected EOF while looking for matching `'''
```

7. Відповіді на контрольні питання

1. **action, method, name, enctype.**
   - `action` — адреса обробника; без нього форма надсилається на поточну адресу.
   - `method` — GET (дані в URL) чи POST (у тілі).
   - `name` — ключ поля в даних запиту; поле без `name` або з `disabled` не надсилається.
   - `enctype` — формат тіла POST: типовий `application/x-www-form-urlencoded` чи `multipart/form-data`, потрібний для файлів.
2. **Суперглобальні масиви.**
   - `$_GET` — параметри рядка запиту, зокрема в URL POST-запиту.
   - `$_POST` — поля тіла POST.
   - `$_FILES` — метадані завантажених файлів.
   - `$_REQUEST` — поєднання GET, POST і, залежно від `request_order`, COOKIE. Джерело значення неочевидне, тому для POST-форми використано `$_POST`.
3. **isset() і тип.** `isset()` підтверджує лише, що ключ існує й значення не `null`. Порожній рядок і масив `login[]=x` його проходять. Тому потрібна окрема перевірка `is_string()` перед `trim()` чи `htmlspecialchars()`, а для обов’язковості — явне `=== ''`.
4. **required.** Це лише перевірка в браузері. Її обходять засобами розробника, вимкненням JavaScript-валідації чи прямим запитом (curl), як у сценаріях 4–5. Сервер мусить повторювати всі перевірки.
5. **Валідація, нормалізація, екранування.** Нормалізація приводить допустиме значення до стандартного вигляду (`trim()`). Валідація вирішує, чи значення допустиме

(довжина, формат, перелік). Екранування робить значення безпечним у конкретному контексті виведення: `htmlspecialchars()` для HTML, але не для SQL чи URL.

6. **Пароль і trim().** Пробіли можуть бути частиною пароля. `trim()` змінив би його, і правильний пароль із пробілом (`taras_m`) перестав би збігатися з хешем. Виводити пароль не можна: він опиниться в HTML, кеші, на знімках і в журналах. Тому поле пароля після помилки порожнє.
7. **password_hash() і password_verify().** `password_hash()` створює bcrypt-хеш з випадковою сіллю; рядок містить алгоритм, вартість і сіль. Тому однаковий пароль дає різні хеші. `password_verify()` бере ці параметри з рядка хешу й порівнює результат за сталий час. MD5 швидкий і без солі, тож вразливий до перебору й таблиць попередньо обчислених хешів.
8. **Масив користувачів.** Зміни потребують редагування коду: немає реєстрації, зміни пароля чи одночасного запису. Файл можна випадково опублікувати разом із хешами. Пошук і розмір обмежені пам'яттю. Немає журналу входів і блокування перебору.
9. **include і require.** За відсутності файла `include` видає попередження й продовжує роботу, а `require` завершує виконання з фатальною помилкою. `__DIR__` дає шлях відносно каталогу самого файла, а не поточного робочого каталогу процесу, тож підключення не залежить від того, звідки запущено сценарій.
10. **$_FILES.** Першим перевіряють `error`: лише за `UPLOAD_ERR_OK` решта ключів має сенс. MIME-тип `type` надсилає клієнт, і він довільний: файл `.jpg` з текстом пройшов як `image/jpeg`. Справжній тип визначають за вмістом (наприклад, `finfo`).
11. **Відсутність вкладення і помилка.** Якщо файл не вибрано, браузер надсилає частину з `filename=""`, і PHP ставить `UPLOAD_ERR_NO_FILE` — це допустимо. Інші ненульові коди (`INI_SIZE`, `PARTIAL` тощо) — помилка передавання, яка блокує успіх. Порожній файл з ім'ям — окремий випадок: код OK, але розмір 0, і це теж помилка.
12. **CSRF.** Чужий сайт може змусити браузер надіслати POST на наш сервер разом із cookies користувача. Метод POST цього не забороняє, а екранування захищає лише від XSS у виведенні. Від CSRF захищає непередбачуваний токен, пов'язаний із сесією, який сервер перевіряє до дії.

8. Висновок

Створено дві форми: перевірку навчальних облікових даних із bcrypt-хешами та зворотний зв'язок із показом метаданих вкладення.

- Сервер перевіряє метод, наявність, типи, обов'язковість, формат і межі. Некоректне введення, масиви й перевищення лімітів не спричиняють попереджень у коді.
- Користувацькі дані виводяться екранованими, пароль і хеш не показуються.

- Обробку відокремлено від розмітки: `index.php` / `login-form.php` / `users.php`, `feedback.php` / `feedback-form.php`.

Усі 15 груп сценаріїв виконано.

Виправлені помилки.

- **Розмітка заготовки `index001.php`.** Список помилок `<ul role="alert">` W3C відхилив: роль `alert` для `ul` недопустима. Виправлено обгорткою `<div role="alert"><ul>…` у формі входу та в обробнику кроку 2.
- **Скрипт перевірок.** Перші запуски показали, що `curl -F` для Windows не бачить шляхи MSYS (`/tmp`), і не вміє надіслати «файл не вибрано», як браузер. Тому використано Windows-шляхи й сире тіло multipart.

Обмеження прикладу:

- вхід перевіряється в межах одного запиту: сесії немає, інші сторінки не захищені;
- немає обмеження кількості спроб, тобто захисту від перебору паролів;
- користувачі зберігаються в PHP-файлі;
- MIME-тип вкладення не перевіряється за вмістом, файл не зберігається;
- немає HTTPS.

Якби приклад став справжнім сервісом, CSRF-токен, пов’язаний із сесією, потрібно було б перевіряти:

- у формі входу — до `password_verify()` і створення сесії, щоб чужий сайт не «залогінив» користувача у свій обліковий запис;
- у формі зворотного зв’язку — до надсилання листа чи збереження повідомлення, щоб чужий сайт не надсилав повідомлень від імені користувача.